Sri Lanka Institute of Information Technology

Machine Learning (IT4060)

Group ID: 95

Group Details:

Student ID	**Student Name**
IT22576170	Kavinda K G D
IT22143204	Fonseka A A M D
IT22298126	Athuraliya D R

Table of Contents

1. Introduction to the Problem 4
2. Problem Statement 5
3. Data Description 6
4. Data Preprocessing 8
5. Methodology / Algorithms Used 10
   5.1 Logistic Regression 10
   5.2 Decision Tree 10
   5.3 Random Forest 11
   5.4 K-Nearest Neighbors (KNN) 11
   5.5 Model Evaluation Approach 12
6. Implementation 13
7. Results 15
8. Discussion 17
9. Limitations and Future Work 19
10. Individual Contribution 21
11. Conclusion 22
12. References 23
13. Apendix 24
   Appendix A – Data Preprocessing Code 24
   Appendix B – Logistic Regression Code 29
   Appendix C – Decision Tree Code 32
   Appendix D – Random Forest Code 35
   Appendix E – K-Nearest Neighbors Code 38
   Appendix F – Model Comparison Code 41

1. Introduction to the Problem

In the food and beverage sector, the quality of the wine has significant importance. The quality of the wine has traditionally been evaluated by experts using sensory evaluation. However, this process may be time-consuming and costly. With the advent of data-driven technologies such as machine learning, the quality of the wine can be efficiently evaluated. Machine learning helps the computer to learn from the data and make predictions. Machine learning can be applied in the food and beverage sector to evaluate the quality of the wine. Machine learning can be applied to evaluate the quality of the wine by examining the physicochemical properties such as the level of acidity, sugar content, pH value, alcohol content, and sulphydrates present in the wine.

This project aims to evaluate the quality of the white wine using machine learning techniques. The real-world dataset was obtained from the UCI Machine Learning Repository. The quality of the wine was evaluated by converting the wine quality score into binary classification in order to predict the quality of the wine as good or poor quality. Four supervised machine learning algorithms were applied in this research: Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors.

The main goal of this project was to compare the performance of these four algorithms and determine the best algorithm for predicting the quality of white wines. This was done by processing the data set and splitting it into training and testing sets. The performance of the algorithms was then evaluated using performance metrics such as accuracy, precision, recall, and F1 score. The findings of this study show the applicability of machine learning in the real world in the domain of food quality.

2. Problem Statement

Wine quality assessment is an important task in the food and beverage industry because it directly affects product value, consumer preference, and market competitiveness. Traditionally, wine quality is evaluated through sensory testing by human experts. Although this method is widely used, it can be expensive, time-consuming, and influenced by subjective judgment. As a result, there is a need for a more efficient and data-driven approach to support quality prediction.

The problem addressed in this project is the prediction of white wine quality using machine learning. The dataset contains several physicochemical properties of white wine samples, such as acidity, sugar content, chlorides, sulphates, density, pH, and alcohol level. These properties can be used as input features to build a predictive model. The target variable in the original dataset is the wine quality score. For the purpose of this study, the target was transformed into a binary classification problem in order to identify whether a wine sample is of good quality or poor quality.

This project aims to determine how effectively supervised machine learning algorithms can classify white wine quality based on its physicochemical characteristics. Four different supervised learning algorithms were selected and applied to the dataset: Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors (KNN). The performance of these models was then compared using standard classification evaluation metrics.

3. Data Description

The data set used in the current project is the White Wine Quality Data Set obtained from the UCI Machine Learning Repository. The data set is a real-world data set that is publicly available. It is used in machine learning problems or experiments. The data set contains the physicochemical test results of Portuguese "Vinho Verde" white wine samples along with their quality ratings. The data set was chosen because it is reliable and well-structured.

The white wine data set used in the current project is composed of 4,898 instances in total with 12 columns. Out of the columns in the data set, 11 columns represent the input variables, while the rest represents the target variable. The target variable is the quality of the wine samples. The input variables represent the numerical physicochemical characteristics of the wine samples.

The attributes included in the dataset are:

- fixed acidity
- volatile acidity
- citric acid
- residual sugar
- chlorides
- free sulfur dioxide
- total sulfur dioxide
- density
- pH
- sulphates
- alcohol
- quality

All input features in the dataset are numeric, which makes the dataset convenient for applying classical machine learning algorithms. Another useful characteristic of this

dataset is that it does not contain missing values, which reduces the amount of data cleaning required during preprocessing. However, the dataset still required further preparation such as checking duplicates, examining class distribution, and transforming the target variable into a more suitable form for binary classification.

The original quality variable is represented as an ordinal score. Since the objective of this project was to solve a supervised classification problem, the target variable was converted into two classes. Wines with a quality score of 6 or above were labeled as good quality, while wines with a quality score below 6 were labeled as poor quality. This transformation simplified the classification problem and made it easier to compare the performance of the selected machine learning algorithms.

The dataset was chosen because it is a real-world dataset from a trusted academic source, contains a sufficient number of records for model training and testing, and includes meaningful numerical attributes that are directly related to the quality of wine. These characteristics made it appropriate for the objectives of this assignment.

4. Data Preprocessing

Data preprocessing is an essential stage in any machine learning project because raw data is not always immediately suitable for model training. The purpose of preprocessing is to understand the structure of the dataset, identify any data quality issues, and transform the data into a form that can be effectively used by machine learning algorithms. In this project, several preprocessing steps were carried out on the White Wine Quality Dataset before training the models.

The first step was to load the dataset into the Python environment and inspect its overall structure. This included checking the number of rows and columns, viewing the first few records, identifying the feature names, and examining the data types of each column. Through this initial inspection, it was confirmed that the dataset contained 4,898 instances and 12 columns, with 11 numerical input features and 1 target variable. Since all input variables were numeric, no categorical encoding was required.

The dataset was then checked for missing values. Missing values can negatively affect model performance if not handled properly. In this study, the dataset did not contain any missing values. However, a missing-value handling strategy was still considered as part of the preprocessing pipeline. If missing values had been found, numerical features would have been imputed using the median, while categorical features would have been filled using the mode. Including this step demonstrated a complete and robust preprocessing approach.

After that, duplicate rows were examined. Duplicate records can create bias in model learning because the same information may be repeated multiple times. The dataset was checked for duplicate entries, and appropriate action was taken to ensure that the training data remained clean and consistent. This helped improve the reliability of the learning process.

The original distribution of the target variable, quality, was also examined. This was an important step because the wine quality scores were not evenly distributed across all classes. Some quality scores appeared more frequently than others, which indicated class imbalance in the original form of the dataset. To simplify the problem and make it more suitable for binary classification, the target variable was transformed into two classes. Wine samples with a quality score of 6 or above were labeled as good quality, while

samples with a score below 6 were labeled as poor quality. This transformation made the classification problem more manageable and more appropriate for comparing multiple supervised learning algorithms.

Summary statistics were then generated for all numerical features. These statistics included measures such as count, mean, standard deviation, minimum, maximum, and quartiles. This step provided a better understanding of the distribution and scale of the variables. It also helped identify that the feature values were not on the same scale. For example, some variables such as chlorides had relatively small values, while others such as total sulfur dioxide had much larger ranges. This observation was important in deciding whether feature scaling was required.

Once the dataset had been cleaned and the target variable had been prepared, the features and target were separated into input variables (X) and output labels (y). The dataset was then divided into training and testing sets using an 80:20 ratio. Stratified sampling was used during this split in order to preserve the class distribution in both the training and testing subsets. This ensured that the evaluation of the models would be fair and representative.

Finally, feature scaling was applied where necessary. Standardization using StandardScaler was performed for Logistic Regression and K-Nearest Neighbors, because these algorithms are sensitive to differences in feature magnitude. The scaler was fitted only on the training data and then applied to the test data in order to avoid data leakage. For Decision Tree and Random Forest, scaling was not required because tree-based algorithms are not significantly affected by feature scale. Both scaled and unscaled processed datasets were saved for use in the corresponding model training notebooks.

Overall, the preprocessing stage ensured that the dataset was clean, well understood, and properly prepared for supervised classification. It also created a consistent foundation for applying and comparing the four selected machine learning algorithms.

5. Methodology / Algorithms Used

This project was done with a supervised machine learning classification approach. After preprocessing the White Wine Quality Dataset, four different classification algorithms were selected and applied to the same dataset to compare their performance. Four different machine learning algorithms were selected and applied to the same dataset to compare their performance. These algorithms include Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors (KNN).

The data was split into training and testing sets. These algorithms were selected based on their popularity and applicability to the given problem. These algorithms are easy to understand and interpret. These algorithms are applicable to numerical data. The following are the roles and reasons why each of these algorithms was selected and applied to this project.

5.1 Logistic Regression

Logistic Regression is a supervised learning algorithm, and it is widely used for classification problems. Despite its name, Logistic Regression is actually intended to estimate the probability of an instance in the dataset belonging to a specific class. The final class is then determined based on this probability. In this project, Logistic Regression was employed to predict whether white wine is of good quality or poor quality.

This algorithm was chosen for this project because it is arguably one of the most popular base models for classification-type problems. The Logistic Regression algorithm is known to be efficient, both in terms of computation and simplicity. Furthermore, it is often employed as a benchmark to compare more advanced models. Since Logistic Regression is known to be affected by the scale of input variables, the standardized dataset was utilized for this algorithm. This allowed it to learn from the input feature values more effectively.

5.2 Decision Tree

Decision Tree is a supervised machine learning algorithm used for prediction purposes by splitting the data into different branches according to their features. This algorithm creates a decision tree in which each node represents the decision rule, the branches represent the outcomes, and the leaf node represents the class label. In this project, the Decision

Tree model was used for classifying the quality of white wine based on the physicochemical properties of the wine samples.

Decision Tree was chosen for the research because it is simple, transparent, and can handle nonlinear data. This algorithm is best used when the decision-making approach is required because the decision can be explained easily. Unlike other algorithms, Decision Tree does not require scaling of the data; hence, the unscaled data was used for training the model.

5.3 Random Forest

Random Forest is an algorithm based on ensemble learning, which combines multiple decision trees to make an accurate prediction. The algorithm is based on creating multiple decision trees with random data and features. The final prediction is based on majority voting. In this project, the Random Forest algorithm was used to make predictions more stable and to improve classification accuracy.

The algorithm was chosen for its excellent performance with structured tabular data. Compared to the performance of a single decision tree, the performance of the Random Forest algorithm is better. The algorithm is also more effective at handling complex relationships. As it is an algorithm based on decision trees, it was trained on the unscaled dataset. The default configuration with 100 trees was used for its stable performance and relatively efficient computation.

5.4 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a supervised learning algorithm in which the prediction of the target variable for a given point in the feature space depends on the majority of the target variable for the K nearest neighbors. KNN does not actually fit the model like the rest of the algorithms. Instead, it stores the data and uses distance calculation to determine the nearest neighbors. KNN was used in this project to classify the wine data based on the similarity in the physicochemical properties.

KNN was chosen for this project because it is an easy and efficient classifier. Moreover, KNN offers the advantage of comparing the performance of the model with both linear and tree-based models. KNN uses distance calculation for prediction. Therefore, feature scaling is critical in KNN. If the features in the dataset are not scaled, the performance of

the model may be compromised because the feature with the larger range may affect the model. Therefore, the standardized dataset was used in the KNN model.

5.5 Model Evaluation Approach

In order to make a fair comparison, all four models were trained and tested on the same dataset. The performance of all models was compared based on classification metrics such as accuracy, precision, recall, and F1 Score. Furthermore, confusion matrices and classification reports were created to compare how well each model classified both good quality and poor quality wine samples. The use of various models and metrics allowed for not only identifying the best-performing model based on its accuracy, but also for identifying the model with the most balanced precision and recall. The methodology allowed for an all-rounded comparison of how effective the chosen supervised machine learning models were for classifying white wine quality.

6. Implementation

The implementation of this project has been carried out by utilizing Python and Jupyter Notebook. These tools have been chosen because they are commonly used in machine learning projects. They provide an easy platform for analyzing the data, developing the model, and visualizing the output.

During the implementation of this project, several Python libraries have been used. Pandas has been used for handling the dataset. Scikit-learn has been used for handling the preprocessing, splitting the dataset into a training set and a testing set, and evaluating the model. Matplotlib and Seaborn have been used for visualizing the output in the form of a confusion matrix and a chart. All the functionality has been provided by these libraries for the efficient implementation of the project.

The project was divided into different sections in the form of different notebooks. This was done in order to maintain a clear workflow. A separate notebook was developed specifically for preprocessing, where data exploration, checking for missing values, checking for duplicates, transformation of the target data, splitting the data into training and testing sets, and scaling were performed. After the preprocessing notebook, different notebooks were developed for each of the four different machine learning models, namely Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors. This made the entire work easier to manage, as each model was implemented separately. A separate notebook was developed for comparing the results of all the models, where the results of all the models were combined, the comparison table was generated, and the best model was determined.

During the preprocessing stage, both the scaled and unscaled versions of the data were stored. This was done because different models have different preprocessing requirements. While the Logistic Regression model and the K-Nearest Neighbors model used the scaled data, the Decision Tree model and the Random Forest model used the unscaled data.

The implementation was also based on a consistent evaluation approach for all the models. This is because the model notebook included data loading, separation of features and targets, model training, prediction, and evaluation using accuracy, precision, recall,

F1-score, confusion matrix, and classification report. This was important because it ensured that a fair comparison was made between the four algorithms used.

Apart from the use of Jupyter Notebooks for local implementation, GitHub was also utilized for the implementation of the project. This is because the repository included files like the dataset files, report folder, README file, and the notebooks. This was important because it ensured the organization of the project while at the same time fulfilling the requirements of the assignment by ensuring the implementation was made public on GitHub.

7. Results

After preprocessing the White Wine Quality Dataset and executing the four supervised learning algorithms, the performance of all models was evaluated on the test dataset. The performance was evaluated based on accuracy, precision, recall, and F1 score. These metrics were chosen for performance evaluation as they give an exhaustive idea of performance compared to accuracy.

The final results obtained from the model comparison are presented in Table 1.

Table 1: Performance Comparison of the Four Models

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.7516	0.8007	0.8298	0.8150
Logistic Regression	0.7427	0.7774	0.8547	0.8142
K-Nearest Neighbors (KNN)	0.7264	0.7762	0.8222	0.7985
Decision Tree	0.6683	0.7453	0.7553	0.7502

Based on the results, it is evident that the highest overall accuracy of 75.16% was recorded by the Random Forest model, making it the best model in terms of overall correctness in classification. In addition, the model recorded the highest precision at 0.8007 and the highest F1-score at 0.8150, which implies that the model was able to maintain a high standard in correctly identifying the positive cases while at the same time minimizing the cases of incorrect classification.

The Logistic Regression model recorded the second highest overall correctness in classification, with an overall accuracy of 74.27%, slightly lower than the highest recorded by the Random Forest model. In addition, the model recorded the highest recall value at 0.8547, indicating that it was highly competent in identifying the wines in the good quality category. The model also recorded an F1-score of 0.8142, which is almost similar to the

highest recorded by the Random Forest model, indicating that it was a highly competitive model in terms of correctness in classification.

K-Nearest Neighbors (KNN) resulted in moderate performance as indicated by an accuracy of 72.64% and F1-score of 0.7985. Although KNN results were lower compared to those of Random Forest and Logistic Regression, it performed moderately and proved the importance of distance-based classification when applied on a large scale.

Decision Tree resulted in the poorest performance compared to all other models as indicated by an accuracy of 66.83% and F1-score of 0.7502. Although it was able to classify the data to a certain extent, its performance was poor compared to other models. This proved that a single decision tree is not effective compared to other models for data classification.

Overall, results have proved that ensemble learning and linear classification are effective compared to a single decision tree for classifying white wine quality data. More so, Random Forest and Logistic Regression have proved to be effective compared to other models for classifying white wine data and are therefore the most suitable models for this study.

If charts and confusion matrix figures are included in the report, they should be placed after this section.

8. Discussion

The results of this study show clear differences in the performance of the four supervised learning algorithms applied to the White Wine Quality Dataset. Among the models tested, Random Forest achieved the best overall performance, while Logistic Regression also produced very strong results. K-Nearest Neighbors showed moderate performance, and Decision Tree was the weakest model among the four.

The best-performing model was Random Forest, which achieved the highest accuracy, precision, and F1-score. This result is reasonable because Random Forest is an ensemble learning method that combines the predictions of many decision trees rather than relying on a single tree. By aggregating multiple trees, the model is able to reduce overfitting and improve generalization. This makes Random Forest particularly effective for structured tabular datasets such as the wine quality dataset used in this project. Its strong precision and F1-score indicate that it was able to maintain a good balance between correctly classifying wine samples and reducing incorrect positive predictions.

Although Random Forest achieved the best overall result, Logistic Regression performed almost equally well. Its F1-score was extremely close to that of Random Forest, and it achieved the highest recall among all models. This suggests that Logistic Regression was especially effective in identifying wine samples belonging to the good-quality class. The strong performance of Logistic Regression indicates that the relationship between the input features and the binary target was sufficiently learnable even with a relatively simple linear classification model. This is an important finding because it shows that high model complexity was not strictly necessary to achieve competitive results on this dataset.

K-Nearest Neighbors (KNN) also produced acceptable results, but it was less effective than the top two models. Since KNN is based on distance calculations, its performance depends heavily on the structure of the feature space and the choice of nearest neighbors. Although scaling was correctly applied before training the KNN model, it may still have been influenced by local variations in the dataset. This may explain why it performed reasonably well but did not surpass Random Forest or Logistic Regression.

The Decision Tree model achieved the lowest performance in the study. A single decision tree can easily become too dependent on the specific patterns present in the training data, which can reduce its ability to generalize well to unseen data. This likely contributed to its

lower accuracy and F1-score compared to Random Forest, which benefits from ensemble averaging across many trees. Even though Decision Tree is highly interpretable and easy to understand, its predictive performance was weaker for this particular classification problem.

Another important point in the discussion is the effect of the target transformation. The original quality variable in the dataset was an ordinal score with multiple values. For this project, it was converted into a binary classification problem by grouping wines into good-quality and poor-quality classes. This decision made the classification task simpler and more practical for comparison across algorithms. However, it also reduced some of the detail contained in the original quality scores. Despite this simplification, the binary approach was appropriate for the objectives of the assignment and allowed the models to be compared more clearly.

The evaluation metrics also provide useful insight beyond overall accuracy. For example, Logistic Regression achieved the highest recall, meaning it was better at identifying good-quality wines. Random Forest, on the other hand, achieved the best overall balance according to the F1-score and also had the highest accuracy and precision. This shows why it is important to use multiple evaluation metrics when comparing classification models. A model with slightly lower accuracy may still be useful if it performs better in recall or precision depending on the context of the problem.

Overall, the discussion shows that Random Forest was the most suitable algorithm for this white wine quality classification task, while Logistic Regression was a very strong alternative. The study also demonstrates that the choice of algorithm, preprocessing strategy, and evaluation metrics all play an important role in building an effective machine learning solution.

9. Limitations and Future Work

Although this project successfully applied four supervised learning algorithms to classify white wine quality, there are several limitations that should be acknowledged. Identifying these limitations is important because it helps explain the boundaries of the current study and highlights possible directions for improvement in future work.

One limitation of the project is that only the **white wine dataset** was used for model development and evaluation. The UCI Wine Quality repository also provides a red wine dataset, but it was not included in this study. As a result, the findings of this project are specific to white wine samples and cannot automatically be generalized to other types of wine without further analysis.

A second limitation is the **binary transformation of the target variable**. The original dataset contains wine quality as an ordinal score with several possible values. In this project, the target variable was converted into two classes: good quality and poor quality. This made the classification problem easier to manage and more suitable for comparison across four algorithms, but it also simplified the original information. Some variation between individual quality scores was therefore lost during this process. Future studies could treat the dataset as a **multiclass classification** problem or even as a **regression** problem in order to preserve more detail from the original target variable.

Another limitation is that the models were trained using mainly **baseline parameter settings**. For example, Random Forest was implemented with a standard number of trees, and KNN was trained with a default neighborhood size. Although these settings were sufficient for a fair comparison, more detailed **hyperparameter tuning** could potentially improve performance. Future work could apply techniques such as grid search or cross-validation to optimize parameters and obtain stronger results.

The study also did not perform advanced **feature selection** or dimensionality reduction. All input variables were used in the training process after preprocessing. While this was acceptable for the dataset size and structure, future work could explore whether removing less important features or selecting only the most influential attributes would improve model efficiency and predictive accuracy.

In addition, the project relied on a single **train-test split** for evaluation. Although stratified sampling was used to preserve class distribution, the results may still vary depending on

how the data is divided. A more robust evaluation could be achieved in future work through **k-fold cross-validation**, which would provide a more comprehensive estimate of model performance across multiple data splits.

Finally, the project focused mainly on comparing classification performance metrics such as accuracy, precision, recall, and F1-score. While these measures are important, future work could expand the analysis by including additional performance indicators, more visualizations, and deeper statistical interpretation of the results. It would also be useful to explore methods for handling any class imbalance more explicitly if needed.

In summary, while this project achieved its objective of comparing four supervised learning algorithms for white wine quality classification, there is still room for further improvement. Future studies could include both white and red wine datasets, preserve the original quality scores, tune model parameters more thoroughly, apply cross-validation, and explore more advanced preprocessing and feature engineering techniques. These improvements could lead to even more reliable and informative results.

10. Individual Contribution

The project was completed as a group assignment with three members, and the work was divided in a way that ensured each member contributed to the machine learning implementation. In accordance with the lecturer's requirement, each group member was responsible for at least one machine learning model. The work was distributed across dataset preparation, preprocessing, model development, evaluation, result comparison, and report preparation.

The first stage of the project involved selecting a suitable real-world dataset, defining the problem, and preparing the data for model training. This included loading the dataset, examining its structure, checking for missing values and duplicates, analyzing the original target distribution, converting the target variable into binary classes, splitting the dataset into training and testing sets, and applying feature scaling where required. This preprocessing stage created the foundation for all four machine learning models.

The model implementation work was then divided among the three members so that each member completed at least one algorithm. Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors were developed and evaluated separately using appropriate performance metrics. After the four models were completed, their outputs were compared in order to identify the best-performing algorithm.

In addition to model development, the group also worked on organizing the GitHub repository, preparing the README file, compiling the final report, and preparing the presentation. The final comparison of results and the preparation of the appendix with source code were also important parts of the contribution process.

A suitable contribution breakdown for a three-member group is shown below and should be updated with the actual names and IDs of your group members:

- **Member 1** – Dataset selection, problem definition, data preprocessing, and **Logistic Regression**
- **Member 2** – **Decision Tree** and **Random Forest**
- **Member 3** – **K-Nearest Neighbors (KNN)**, model comparison, GitHub repository organization, README preparation, and report compilation

11. Conclusion

This project successfully applied supervised machine learning techniques to the problem of white wine quality classification. Using the White Wine Quality Dataset from the UCI Machine Learning Repository, the study explored how physicochemical properties of wine can be used to predict whether a wine sample is of good quality or poor quality. The project followed a complete machine learning workflow, including dataset exploration, preprocessing, model training, evaluation, and comparison.

Four supervised learning algorithms were implemented in this study: Logistic Regression, Decision Tree, Random Forest, and K-Nearest Neighbors. Each model was trained and tested using the same prepared dataset, and their performance was evaluated using accuracy, precision, recall, and F1-score. This allowed a fair comparison of the strengths and weaknesses of the selected algorithms.

The results showed that **Random Forest** achieved the best overall performance, with the highest accuracy, precision, and F1-score. **Logistic Regression** also performed very strongly and achieved the highest recall, showing that it was highly effective in identifying good-quality wine samples. K-Nearest Neighbors produced moderate results, while Decision Tree had the lowest performance among the four models. Based on the comparative analysis, Random Forest was identified as the most suitable model for this classification task.

Overall, this project demonstrates that machine learning can be effectively used to support wine quality assessment using measurable physicochemical data. It also shows the importance of proper preprocessing, suitable algorithm selection, and the use of multiple evaluation metrics when solving real-world classification problems. The findings of this study highlight the practical value of supervised learning in quality prediction tasks and provide a strong example of how different machine learning algorithms can be compared in a structured and meaningful way.

12. References

P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. "Wine Quality," UCI Machine Learning Repository, 2009. [Online]. Available: https://doi.org/10.24432/C56S3T.

13. Apendix

Appendix A – Data Preprocessing Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


# ------------------------------
# 1. LOAD DATASET
# ------------------------------
file_path = "D:/ML-Assignment/ML_assignment/data/winequality-white.csv"
df = pd.read_csv(file_path, sep=";")
```

```
# ------------------------------
# 2. UNDERSTAND THE DATASET
# ------------------------------
print("\n===== DATASET OVERVIEW =====")
print("Shape of dataset:", df.shape)

print("\nColumns:")
print(df.columns.tolist())

print("\nFirst 5 rows:")
print(df.head())

print("\nData types:")
print(df.dtypes)
```

```
# ------------------------------
# 3. CHECK MISSING VALUES
# ------------------------------
print("\n===== MISSING VALUES =====")
missing_values = df.isnull().sum()
print(missing_values)

total_missing = missing_values.sum()
print("\nTotal missing values in dataset:", total_missing)
```

```
# Handle missing values if any exist
if total_missing > 0:
    print("\nMissing values found. Handling missing values...")

    for column in df.columns:
        if df[column].isnull().sum() > 0:
            if df[column].dtype in ["int64", "float64"]:
                # For numeric columns, fill with median
                df[column].fillna(df[column].median(), inplace=True)
                print(f"Filled missing values in numeric column '{column}' with
median.")
            else:
                # For categorical columns, fill with mode
                df[column].fillna(df[column].mode()[0], inplace=True)
                print(f"Filled missing values in categorical column '{column}'
with mode.")
else:
    print("\nNo missing values found. No imputation needed.")
```

```
# ------------------------------
# 4. CHECK DUPLICATE ROWS
# ------------------------------
duplicate_count = df.duplicated().sum()
print("\n===== DUPLICATE ROWS =====")
print("Number of duplicate rows:", duplicate_count)

# Optional: remove duplicates
if duplicate_count > 0:
    df = df.drop_duplicates()
    print("Duplicate rows removed.")
    print("New shape after removing duplicates:", df.shape)
else:
    print("No duplicate rows found.")
```

```
# ------------------------------
# 5. CHECK ORIGINAL TARGET DISTRIBUTION
# ------------------------------
print("\n===== ORIGINAL TARGET DISTRIBUTION (quality) =====")
```

```
print(df["quality"].value_counts().sort_index())
```

```
# ------------------------------
# 6. SUMMARY STATISTICS
# ------------------------------
print("\n===== SUMMARY STATISTICS =====")
print(df.describe())
```

```
# ------------------------------
# 7. CONVERT TARGET TO BINARY CLASSIFICATION
#    quality >= 6  -> 1 (Good)
#    quality < 6   -> 0 (Poor)
# ------------------------------
df["quality_label"] = df["quality"].apply(lambda x: 1 if x >= 6 else 0)

print("\n===== BINARY TARGET DISTRIBUTION =====")
print(df["quality_label"].value_counts().sort_index())
```

```
# ------------------------------
# 8. DEFINE FEATURES (X) AND TARGET (y)
# ------------------------------
X = df.drop(columns=["quality", "quality_label"])
y = df["quality_label"]

print("\n===== FEATURES AND TARGET =====")
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
# ------------------------------
# 9. TRAIN-TEST SPLIT
# ------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

```
)

print("\n===== TRAIN-TEST SPLIT =====")
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
# ------------------------------
# 10. FEATURE SCALING
#    Fit only on training data to avoid data leakage
# ------------------------------
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\n===== SCALING COMPLETED =====")
print("Scaled X_train shape:", X_train_scaled.shape)
print("Scaled X_test shape:", X_test_scaled.shape)
```

```
# ------------------------------
# 11. SAVE PROCESSED FILES
# ------------------------------
save_files = True

if save_files:
    # Save unscaled train/test sets
    train_df = pd.concat(
        [X_train.reset_index(drop=True), y_train.reset_index(drop=True)],
        axis=1
    )
    test_df = pd.concat(
        [X_test.reset_index(drop=True), y_test.reset_index(drop=True)],
        axis=1
    )

    train_df.to_csv("D:/ML-
Assignment/ML_assignment/data/white_wine_train_unscaled.csv", index=False)
```

```
    test_df.to_csv("D:/ML-
Assignment/ML_assignment/data/white_wine_test_unscaled.csv", index=False)

    # Save scaled train/test sets
    X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X.columns)
    X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X.columns)

    train_scaled_df = pd.concat(
        [X_train_scaled_df.reset_index(drop=True),
y_train.reset_index(drop=True)],
        axis=1
    )
    test_scaled_df = pd.concat(
        [X_test_scaled_df.reset_index(drop=True),
y_test.reset_index(drop=True)],
        axis=1
    )

    train_scaled_df.to_csv("D:/ML-
Assignment/ML_assignment/data/white_wine_train_scaled.csv", index=False)
    test_scaled_df.to_csv("D:/ML-
Assignment/ML_assignment/data/white_wine_test_scaled.csv", index=False)

    print("\nProcessed files saved successfully in the data folder.")


print("\n===== PREPROCESSING FINISHED =====")
```

Appendix B – Logistic Regression Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# -------------------------------
# 1. LOAD PREPROCESSED SCALED DATA
# -------------------------------
train_path = "../data/white_wine_train_scaled.csv"
test_path = "../data/white_wine_test_scaled.csv"

train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)

print("===== DATA LOADING =====")
print("Train dataset shape:", train_df.shape)
print("Test dataset shape:", test_df.shape)

print("\nFirst 5 rows of training data:")
print(train_df.head())
```

```
# -------------------------------
# 2. SEPARATE FEATURES (X) AND TARGET (y)
# -------------------------------
X_train = train_df.drop(columns=["quality_label"])
y_train = train_df["quality_label"]

X_test = test_df.drop(columns=["quality_label"])
y_test = test_df["quality_label"]
```

```python
print("\n===== FEATURES AND TARGET =====")
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```python
# ------------------------------
# 3. TRAIN LOGISTIC REGRESSION MODEL
# ------------------------------
model = LogisticRegression(random_state=42, max_iter=1000)
model.fit(X_train, y_train)

print("\nLogistic Regression model trained successfully.")
```

```python
# ------------------------------
# 4. MAKE PREDICTIONS
# ------------------------------
y_pred = model.predict(X_test)

print("\nFirst 10 predictions:")
print(y_pred[:10])
```

```python
# ------------------------------
# 5. EVALUATE THE MODEL
# ------------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\n===== LOGISTIC REGRESSION RESULTS =====")
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}")

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```python
# ------------------------------
```

```python
# 6. PLOT CONFUSION MATRIX
# ------------------------------
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Poor", "Good"],
    yticklabels=["Poor", "Good"]
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```

Appendix C – Decision Tree Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# -------------------------------
# 1. LOAD PREPROCESSED UNSCALED DATA
# -------------------------------
train_path = "../data/white_wine_train_unscaled.csv"
test_path = "../data/white_wine_test_unscaled.csv"

train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)

print("===== DATA LOADING =====")
print("Train dataset shape:", train_df.shape)
print("Test dataset shape:", test_df.shape)

print("\nFirst 5 rows of training data:")
print(train_df.head())
```

```
# -------------------------------
# 2. SEPARATE FEATURES (X) AND TARGET (y)
# -------------------------------
X_train = train_df.drop(columns=["quality_label"])
y_train = train_df["quality_label"]

X_test = test_df.drop(columns=["quality_label"])
```

```
y_test = test_df["quality_label"]

print("\n===== FEATURES AND TARGET =====")
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
# ------------------------------
# 3. TRAIN DECISION TREE MODEL
# ------------------------------
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

print("\nDecision Tree model trained successfully.")
```

```
# ------------------------------
# 4. MAKE PREDICTIONS
# ------------------------------
y_pred = model.predict(X_test)

print("\nFirst 10 predictions:")
print(y_pred[:10])
```

```
# ------------------------------
# 5. EVALUATE THE MODEL
# ------------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\n===== DECISION TREE RESULTS =====")
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}")

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
# ------------------------------
# 6. PLOT CONFUSION MATRIX
# ------------------------------
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=["Poor", "Good"],
    yticklabels=["Poor", "Good"]
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Decision Tree")
plt.show()
```

Appendix D – Random Forest Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# -------------------------------
# 1. LOAD PREPROCESSED UNSCALED DATA
# -------------------------------
train_path = "../data/white_wine_train_unscaled.csv"
test_path = "../data/white_wine_test_unscaled.csv"

train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)

print("===== DATA LOADING =====")
print("Train dataset shape:", train_df.shape)
print("Test dataset shape:", test_df.shape)

print("\nFirst 5 rows of training data:")
print(train_df.head())
```

```
# -------------------------------
# 2. SEPARATE FEATURES (X) AND TARGET (y)
# -------------------------------
X_train = train_df.drop(columns=["quality_label"])
y_train = train_df["quality_label"]

X_test = test_df.drop(columns=["quality_label"])
y_test = test_df["quality_label"]
```

```
print("\n===== FEATURES AND TARGET =====")
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
# ------------------------------
# 3. TRAIN RANDOM FOREST MODEL
# ------------------------------
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
model.fit(X_train, y_train)

print("\nRandom Forest model trained successfully.")
```

```
# ------------------------------
# 4. MAKE PREDICTIONS
# ------------------------------
y_pred = model.predict(X_test)

print("\nFirst 10 predictions:")
print(y_pred[:10])
```

```
# ------------------------------
# 5. EVALUATE THE MODEL
# ------------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\n===== RANDOM FOREST RESULTS =====")
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}")

print("\nConfusion Matrix:")
print(cm)
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
# ------------------------------
# 6. PLOT CONFUSION MATRIX
# ------------------------------
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Oranges",
    xticklabels=["Poor", "Good"],
    yticklabels=["Poor", "Good"]
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Random Forest")
plt.show()
```

Appendix E – K-Nearest Neighbors Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# -------------------------------
# 1. LOAD PREPROCESSED SCALED DATA
# -------------------------------
train_path = "../data/white_wine_train_scaled.csv"
test_path = "../data/white_wine_test_scaled.csv"

train_df = pd.read_csv(train_path)
test_df = pd.read_csv(test_path)

print("===== DATA LOADING =====")
print("Train dataset shape:", train_df.shape)
print("Test dataset shape:", test_df.shape)

print("\nFirst 5 rows of training data:")
print(train_df.head())
```

```
# -------------------------------
# 2. SEPARATE FEATURES (X) AND TARGET (y)
# -------------------------------
X_train = train_df.drop(columns=["quality_label"])
y_train = train_df["quality_label"]

X_test = test_df.drop(columns=["quality_label"])
y_test = test_df["quality_label"]
```

```python
print("\n===== FEATURES AND TARGET =====")
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```python
# ------------------------------
# 3. TRAIN KNN MODEL
# ------------------------------
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

print("\nKNN model trained successfully.")
```

```python
# ------------------------------
# 4. MAKE PREDICTIONS
# ------------------------------
y_pred = model.predict(X_test)

print("\nFirst 10 predictions:")
print(y_pred[:10])
```

```python
# ------------------------------
# 5. EVALUATE THE MODEL
# ------------------------------
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\n===== KNN RESULTS =====")
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}")

print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```python
# ------------------------------
```

```python
# 6. PLOT CONFUSION MATRIX
# ------------------------------
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Purples",
    xticklabels=["Poor", "Good"],
    yticklabels=["Poor", "Good"]
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - KNN")
plt.show()
```

Appendix F – Model Comparison Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# ------------------------------
# 1. LOAD PREPROCESSED DATA
# ------------------------------
# Scaled data for Logistic Regression and KNN
train_scaled_path = "../data/white_wine_train_scaled.csv"
test_scaled_path = "../data/white_wine_test_scaled.csv"

train_scaled_df = pd.read_csv(train_scaled_path)
test_scaled_df = pd.read_csv(test_scaled_path)

# Unscaled data for Decision Tree and Random Forest
train_unscaled_path = "../data/white_wine_train_unscaled.csv"
test_unscaled_path = "../data/white_wine_test_unscaled.csv"

train_unscaled_df = pd.read_csv(train_unscaled_path)
test_unscaled_df = pd.read_csv(test_unscaled_path)

print("Scaled train shape:", train_scaled_df.shape)
print("Scaled test shape:", test_scaled_df.shape)
print("Unscaled train shape:", train_unscaled_df.shape)
```

```
print("Unscaled test shape:", test_unscaled_df.shape)
```

```
# ------------------------------
# 2. PREPARE X AND y
# ------------------------------
# For Logistic Regression and KNN
X_train_scaled = train_scaled_df.drop(columns=["quality_label"])
y_train_scaled = train_scaled_df["quality_label"]

X_test_scaled = test_scaled_df.drop(columns=["quality_label"])
y_test_scaled = test_scaled_df["quality_label"]

# For Decision Tree and Random Forest
X_train_unscaled = train_unscaled_df.drop(columns=["quality_label"])
y_train_unscaled = train_unscaled_df["quality_label"]

X_test_unscaled = test_unscaled_df.drop(columns=["quality_label"])
y_test_unscaled = test_unscaled_df["quality_label"]
```

```
# ------------------------------
# 3. DEFINE MODELS
# ------------------------------
models = {
    "Logistic Regression": {
        "model": LogisticRegression(random_state=42, max_iter=1000),
        "X_train": X_train_scaled,
        "y_train": y_train_scaled,
        "X_test": X_test_scaled,
        "y_test": y_test_scaled
    },
    "Decision Tree": {
        "model": DecisionTreeClassifier(random_state=42),
        "X_train": X_train_unscaled,
        "y_train": y_train_unscaled,
        "X_test": X_test_unscaled,
        "y_test": y_test_unscaled
    },
    "Random Forest": {
        "model": RandomForestClassifier(n_estimators=100, random_state=42),
        "X_train": X_train_unscaled,
        "y_train": y_train_unscaled,
        "X_test": X_test_unscaled,
```

```
        "y_test": y_test_unscaled
    },
    "KNN": {
        "model": KNeighborsClassifier(n_neighbors=5),
        "X_train": X_train_scaled,
        "y_train": y_train_scaled,
        "X_test": X_test_scaled,
        "y_test": y_test_scaled
    }
}
```

```
# ------------------------------
# 4. TRAIN, PREDICT, EVALUATE
# ------------------------------
results = []

for model_name, config in models.items():
    print(f"\n===== {model_name} =====")

    model = config["model"]
    X_train = config["X_train"]
    y_train = config["y_train"]
    X_test = config["X_test"]
    y_test = config["y_test"]

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    print(f"Accuracy  : {accuracy:.4f}")
    print(f"Precision : {precision:.4f}")
    print(f"Recall    : {recall:.4f}")
    print(f"F1-Score  : {f1:.4f}")

    print("\nConfusion Matrix:")
    print(cm)
```

```
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred))

    results.append({
        "Model": model_name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    })
```

```
# ------------------------------
# 5. CREATE COMPARISON TABLE
# ------------------------------
results_df = pd.DataFrame(results)

print("\n===== MODEL COMPARISON TABLE =====")
print(results_df)

# Optional: sort by accuracy
results_df = results_df.sort_values(by="Accuracy",
ascending=False).reset_index(drop=True)

print("\n===== SORTED MODEL COMPARISON TABLE =====")
print(results_df)
```

```
# ------------------------------
# 6. SAVE COMPARISON TABLE
# ------------------------------
results_df.to_csv("../data/model_comparison_results.csv", index=False)
print("\nComparison results saved to ../data/model_comparison_results.csv")
```

```
# ------------------------------
# 7. PLOT COMPARISON CHARTS
# ------------------------------
plt.figure(figsize=(10, 6))
sns.barplot(x="Model", y="Accuracy", data=results_df)
plt.title("Model Accuracy Comparison")
plt.ylim(0, 1)
plt.xticks(rotation=15)
plt.show()
```

```
plt.figure(figsize=(10, 6))
sns.barplot(x="Model", y="F1-Score", data=results_df)
plt.title("Model F1-Score Comparison")
plt.ylim(0, 1)
plt.xticks(rotation=15)
plt.show()
```

```
# ------------------------------
# 8. IDENTIFY BEST MODEL
# ------------------------------
best_model = results_df.iloc[0]

print("\n===== BEST MODEL =====")
print(f"Best Model   : {best_model['Model']}")
print(f"Accuracy     : {best_model['Accuracy']:.4f}")
print(f"Precision    : {best_model['Precision']:.4f}")
print(f"Recall       : {best_model['Recall']:.4f}")
print(f"F1-Score     : {best_model['F1-Score']:.4f}")
```

```
# ------------------------------
# 9. SHORT CONCLUSION
# ------------------------------
print("\n===== CONCLUSION =====")
print(
    f"Among the four supervised learning algorithms tested, "
    f"{best_model['Model']} achieved the best overall performance "
    f"based on accuracy. This suggests that it is the most suitable "
    f"model for predicting white wine quality in this study."
)
```